

DOCUMENTACAO DE PROJETO

Sistema de Gerenciamento de Tarefas com Autenticacao

Mateus da Silveira Batista

Engenheiro de Software | Desenvolvedor Web

Maio de 2026

1. Visao Geral do Projeto

Este projeto tem como objetivo retomar e consolidar habilidades de desenvolvimento web fullstack apos um periodo de afastamento da pratica. A aplicacao escolhida e um Sistema de Gerenciamento de Tarefas com autenticacao, cobrindo todos os pilares essenciais de uma aplicacao moderna em producao.

1.1 Objetivos

- Reativar o raciocinio de modelagem de banco de dados relacional com Prisma e PostgreSQL
- Praticar criacao e consumo de APIs REST tipadas com TypeScript e Zod
- Dominar o App Router do Next.js 14+ (Server Components vs Client Components)
- Implementar autenticacao OAuth real com NextAuth.js
- Publicar aplicacao em producao com deploy na Vercel

1.2 Stack Tecnologica

Categoria	Tecnologia	Justificativa
Framework	Next.js 14+ (App Router)	Padrao do mercado; cobre frontend e backend em um so projeto
Linguagem	TypeScript	Ja utilizado nos projetos anteriores; reforca tipagem em toda a stack
ORM / Banco	Prisma + PostgreSQL (Neon)	Abstrai SQL sem esconder a logica; Neon oferece Postgres gratuito

Autenticacao	NextAuth.js (Auth.js v5)	Solucao completa com OAuth; evita implementar auth do zero
Validacao	Zod	Padrao atual para validar dados em endpoints e formularios
Estilizacao	Tailwind CSS	Ja dominado; mantém foco no que e novo
Deploy	Vercel + Neon	Integracao nativa com Next.js; ambos gratuitos para projetos pessoais

2. Arquitetura da Aplicacao

2.1 Modelo de Dados

Duas entidades principais, com relacionamento direto (um usuario tem muitas tarefas):

Entidade User

Campo	Tipo	Descricao
id	String (cuid)	Chave primaria gerada automaticamente
name	String?	Nome do usuario (vindo do OAuth)
email	String (unique)	Email unico, usado como identificador
image	String?	URL do avatar (Google/GitHub)
createdAt	DateTime	Data de criacao do registro

Entidade Task

Campo	Tipo	Descricao
id	String (cuid)	Chave primaria
title	String	Titulo da tarefa (obrigatorio)
description	String?	Descricao opcional
status	Enum	TODO IN_PROGRESS DONE
priority	Enum	LOW MEDIUM HIGH
userId	String (FK)	Referencia ao User dono da tarefa
createdAt	DateTime	Data de criacao
updatedAt	DateTime	Ultima atualizacao (auto)

2.2 Estrutura de Pastas

Pasta / Arquivo	Finalidade
app/	Raiz do App Router do Next.js
app/api/tasks/	Endpoints REST das tarefas (GET, POST)
app/api/tasks/[id]/	Endpoints com ID (PATCH, DELETE)
app/(auth)/	Paginas de login/cadastro (route group)
app/dashboard/	Pagina principal logada com listagem de tarefas
components/	Componentes React reutilizaveis
lib/	Configuracao do Prisma, Auth, helpers
prisma/schema.prisma	Definicao do modelo de dados
middleware.ts	Protecao de rotas autenticadas

2.3 Endpoints da API

Metodo	Rota	Descricao	Auth?
GET	/api/tasks	Lista tarefas do usuario logado	Sim
POST	/api/tasks	Cria nova tarefa	Sim
PATCH	/api/tasks/[id]	Atualiza status ou prioridade	Sim
DELETE	/api/tasks/[id]	Remove tarefa	Sim

3. Cronograma Detalhado

Planejado para 2 a 3 semanas, dedicando 1 a 2 horas por dia. Cada fase tem um entregavel concreto para validar o progresso antes de avancar.

Fase 1 — Setup e Banco de Dados (Dias 1 a 4)

Entregavel: banco criado no Neon, schema definido, migrations rodando e dados de seed visiveis no Prisma Studio.

Dia	Tarefa	Conceito-chave
1	Criar projeto com create-next-app (TypeScript + App Router + Tailwind)	Setup do ambiente, estrutura de pastas do Next.js
2	Criar conta no Neon, conectar Prisma, configurar DATABASE_URL no .env	Variavel de ambiente, string de conexao PostgreSQL
3	Escrever schema.prisma com User e Task; rodar prisma migrate dev	Modelagem relacional, migrations, enums
4	Criar seed.ts com dados de teste; explorar Prisma Studio	Script de seed, inspecao visual do banco

Fase 2 — API REST (Dias 5 a 9)

Entregavel: todos os endpoints funcionando e testados via Thunder Client ou Insomnia.

Dia	Tarefa	Conceito-chave
5	Criar GET /api/tasks e POST /api/tasks com validacao Zod	Route Handlers no App Router, validacao de entrada
6	Criar PATCH /api/tasks/[id] para atualizar status e prioridade	Rotas dinamicas, atualizacao parcial
7	Criar DELETE /api/tasks/[id] com verificacao de ownership	Seguranca: impedir deletar tarefa de outro usuario
8	Adicionar tratamento de erros padronizado (400, 401, 404, 500)	HTTP status codes, respostas consistentes
9	Testar todos os endpoints com Thunder Client; corrigir bugs	Ferramentas de teste de API, debugging

Fase 3 — Autenticacao (Dias 10 a 13)

Entregavel: login com Google funcionando; usuario so acessa as proprias tarefas.

Dia	Tarefa	Conceito-chave
10	Instalar e configurar NextAuth.js com provider Google (Google Cloud Console)	OAuth2, credenciais de cliente, callback URL
11	Criar middleware.ts para proteger rotas do dashboard	Middleware do Next.js, sessao do servidor
12	Integrar session nos endpoints da API (validar usuario logado)	getSession, contexto de autenticacao
13	Criar pagina de login com botao Sign in with Google	Client Components, hooks de sessao (getSession)

Fase 4 — Frontend (Dias 14 a 18)

Entregavel: dashboard completo, responsivo e funcionando com dados reais do banco.

Dia	Tarefa	Conceito-chave
14	Criar layout do dashboard: header com avatar, sidebar e area de conteudo	Layout compartilhado, componentes de servidor
15	Listar tarefas com filtro por status (TODO / IN_PROGRESS / DONE)	Server Components buscando dados direto no banco
16	Criar modal/form de adicao de tarefa com validacao no cliente	Client Component, useState, chamada fetch a API
17	Adicionar acoes inline: mudar status, deletar tarefa com confirmacao	Optimistic UI ou re-fetch pos-acao
18	Revisar responsividade, polir UI com Tailwind	Mobile-first, classes responsivas do Tailwind

Fase 5 — Qualidade e Deploy (Dias 19 a 21)

Entregavel: aplicacao publicada na Vercel com URL publica e README documentado.

Dia	Tarefa	Conceito-chave
19	Configurar ESLint + Prettier; corrigir todos os warnings	Qualidade de codigo, padronizacao
20	Escrever 1 teste de integracao para GET /api/tasks com Vitest	Teste de API, mocking do Prisma
21	Deploy na Vercel; configurar env vars no painel; testar em producao	Deploy, variaveis de ambiente em producao, DNS

4. Recursos de Estudo por Fase

Lista curada por topico. Priorize a documentacao oficial sempre que existir em bom estado — ela e a fonte mais atualizada. Os videos do YouTube sao indicados para quem precisa ver o conceito em acao antes de ler.

4.1 Next.js 14+ e App Router

Este é o maior ponto de atualização. O App Router mudou fundamentalmente como páginas e dados funcionam.

- Documentação oficial (obrigatória): nextjs.org/docs — comece por 'Building Your Application > Routing'
- YouTube — Fireship: 'Next.js 13... is it good?' (13 min) — visão geral rápida das mudanças
- YouTube — Traversy Media: 'Next.js 14 Crash Course' — projeto do zero com App Router
- YouTube — ByteGrad: canal dedicado a Next.js com App Router; episódios curtos por tópico

4.2 Prisma e PostgreSQL

- Documentação oficial: prisma.io/docs — seções 'Get started' e 'Prisma schema'
- YouTube — Fireship: 'Prisma in 100 seconds' — overview rápido do conceito
- YouTube — Web Dev Simplified: 'Learn Prisma in 60 minutes' — cobre schema, migrations e queries
- Neon (banco gratuito): neon.tech — documentação de conexão com Prisma está na própria dashboard

4.3 TypeScript e Zod

- TypeScript — Documentação oficial: typescriptlang.org/docs (focar em Generics e Utility Types)
- Zod — Documentação oficial: zod.dev — é curta e direta; leia inteira antes de começar a Fase 2
- YouTube — Matt Pocock: canal 'Total TypeScript' — melhor recurso atual para TS avançado
- YouTube — Fireship: 'Zod in 15 minutes' — suficiente para começar a usá-lo

4.4 NextAuth.js (Auth.js v5)

- Documentação oficial: authjs.dev — use esta, não a antiga next-auth.js.org (desatualizada)
- YouTube — Code With Antonio: 'Next Auth v5 Tutorial' — o mais completo disponível com App Router
- Google Cloud Console: console.cloud.google.com — necessário para criar credenciais OAuth

4.5 Testes com Vitest

- Documentacao oficial: vitest.dev — simples e rapida de ler
- YouTube — Leandro Ercole: 'Vitest - Testes unitarios no Next.js' — em portugues
- Prisma docs: 'Unit Testing' — mostra como mockar o PrismaClient nos testes

4.6 Deploy e Infraestrutura

- Vercel docs: vercel.com/docs — secao 'Frameworks > Next.js' explica deploy automatico via Git
- Neon docs: neon.tech/docs — secao 'Connection pooling' importante para producao

5. Boas Praticas e Pontos de Atencao

5.1 Diferenca critica: Server vs Client Components

Este e o conceito que mais gera confusao para quem esta retomando o Next.js. A regra geral:

Server Component	Client Component
Padrao no App Router; nao precisa declarar nada	Requer 'use client' no topo do arquivo
Pode buscar dados diretamente (async/await)	Nao pode ser async; usa hooks (useState, useEffect)
Nao pode usar hooks do React nem eventos do browser	Pode usar eventos (onClick) e estado local
Melhor para SEO e performance inicial	Necessario para interatividade do usuario

5.2 Seguranca basica nos endpoints

- Sempre verificar se o usuario esta autenticado antes de qualquer operacao no banco
- Nunca confiar no userId que vem do corpo da request — sempre pegar da sessao do servidor
- Validar todos os dados de entrada com Zod antes de tocar no banco
- Verificar se a tarefa pertence ao usuario antes de atualizar ou deletar

5.3 Variaveis de ambiente

Variavel	Onde usar
DATABASE_URL	String de conexao do Neon (nunca commitar no Git)
NEXTAUTH_SECRET	Chave secreta do NextAuth (gerar com: <code>openssl rand -base64 32</code>)
NEXTAUTH_URL	URL da aplicacao (http://localhost:3000 em dev; URL da Vercel em prod)

GOOGLE_CLIENT_ID	Credencial do Google Cloud Console
GOOGLE_CLIENT_SECRET	Credencial do Google Cloud Console

5.4 Git — fluxo recomendado

- Criar um commit por fase concluída com mensagem clara (ex: 'feat: add task CRUD endpoints')
- Manter .env no .gitignore desde o início — nunca commitar credenciais
- Criar .env.example com as variáveis sem valores para documentar o projeto

6. Critérios de Conclusão

O projeto está completo quando todos os itens abaixo estiverem marcados:

Item	Descrição
Banco	Schema criado, migrations aplicadas, seed funcionando
API	4 endpoints (GET, POST, PATCH, DELETE) funcionando com validação Zod
Auth	Login com Google funcionando; rotas protegidas por middleware
Frontend	Dashboard com listagem, criação, atualização e exclusão de tarefas
Segurança	Usuário só vê e edita as próprias tarefas
Qualidade	ESLint sem erros; Prettier configurado; 1 teste de integração passando
Deploy	Aplicação publicada na Vercel com URL pública acessível
Documentação	README com instruções de setup, stack utilizada e link do deploy

Ao concluir este projeto, o portfólio contará com uma aplicação fullstack real com autenticação, banco relacional, API tipada e deploy em produção — cobrindo exatamente o que vagas de desenvolvedor mid-level exigem no mercado atual.